AI Powered Intelligent Hospital Patient Flow Management System

DESIGN AND IMPLEMENTATION

Student	Hall Ticket No.
A. RAMU	23K91A6715
B. RUSHI	23K91A6720
B. RAJ KUMAR	23K91A6725
A. PRABHAS	23K91A6703

Under the guidance of [GUIDE NAME]

Department of Computer Science and Engineering (Data Science)

[COLLEGE NAME]

[ACADEMIC YEAR]

1. Design Goals

The design prioritises a single source of operational state, role separation, visible queue ordering, traceable movement and small replaceable modules. Predictions are computed at the edge of visit creation and stored with the visit so every dashboard presents the same result.

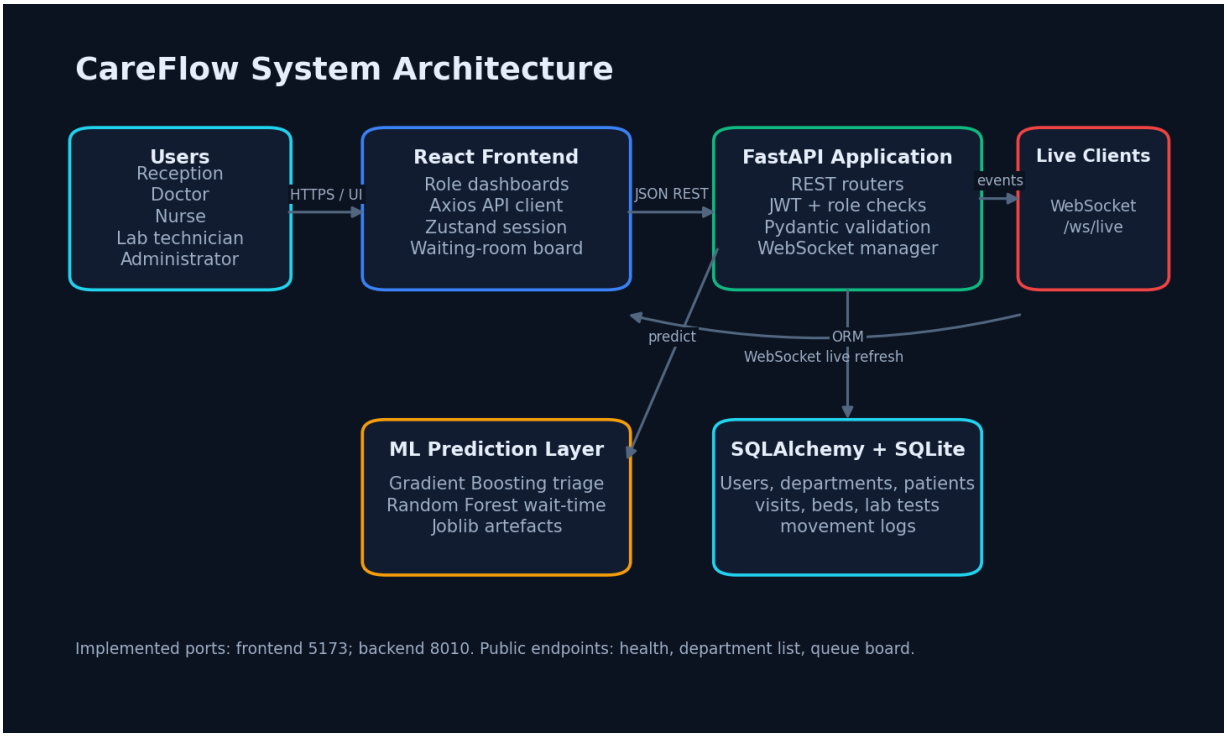


Figure 1. Implemented architecture.

2. Module Breakdown

Table 1. Backend modules.

Module	Responsibility
api/auth.py	Login and current-user endpoint
api/visits.py	Registration, queue, assignment, state and movement
api/beds.py	Capacity, admission and release
api/labs.py	Laboratory orders and progress
api/dashboard.py	KPIs, load and public queue
services/queue_service.py	Token and priority ordering
services/ws_manager.py	In-process WebSocket broadcast
ml/predictor.py	Model loading and inference

3. Database Design

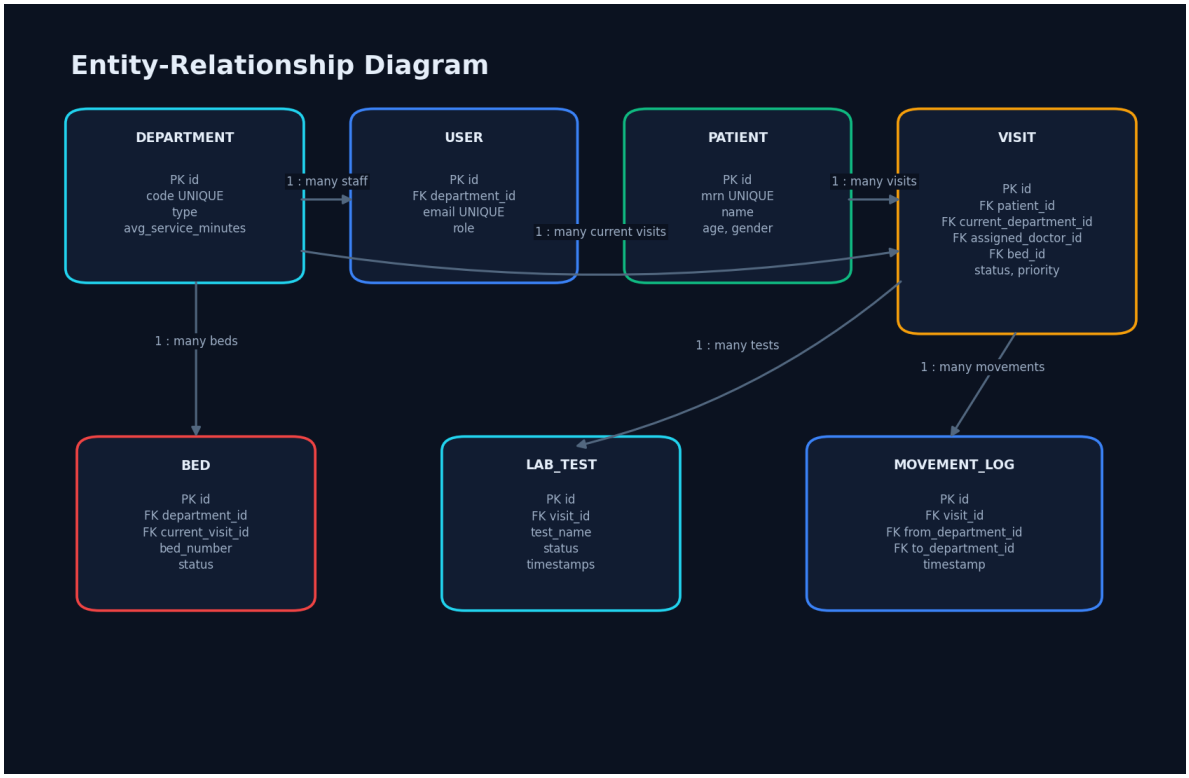


Figure 2. Database entity relationships.

Seven SQLAlchemy entities persist staff, departments, patients, visits, beds, laboratory tests and movement logs. String identifiers are generated from UUID values. Enumerations constrain workflow states. Relationships support role context, patient history, department capacity and visit-linked work.

4. Visit State Design

VisitStatusEnum includes waiting, in_consultation, lab_pending, lab_in_progress, pharmacy, admitted_ward, admitted_icu, discharged and cancelled. Priority is critical, urgent or normal. Queue sorting uses priority rank and token number; a critical new visit has zero patients counted ahead.

MovementLog is created at registration and when a status update includes a new department. This makes location transitions inspectable, although admission currently changes the visit department without adding a movement record.

5. API Design

Table 2. API design summary.

Group	Principal methods	Protection
Authentication	POST login; GET me	Public login / bearer token
Visits	POST, GET, PATCH status, PATCH doctor	Role checks
Beds	GET, PATCH, POST admit/release	Authenticated; write roles
Labs	POST, GET, PATCH	Doctor/admin or lab/admin
Dashboard	GET KPI/load/queue	Authenticated except queue board
Live	WebSocket /ws/live	No authentication

6. ML Integration

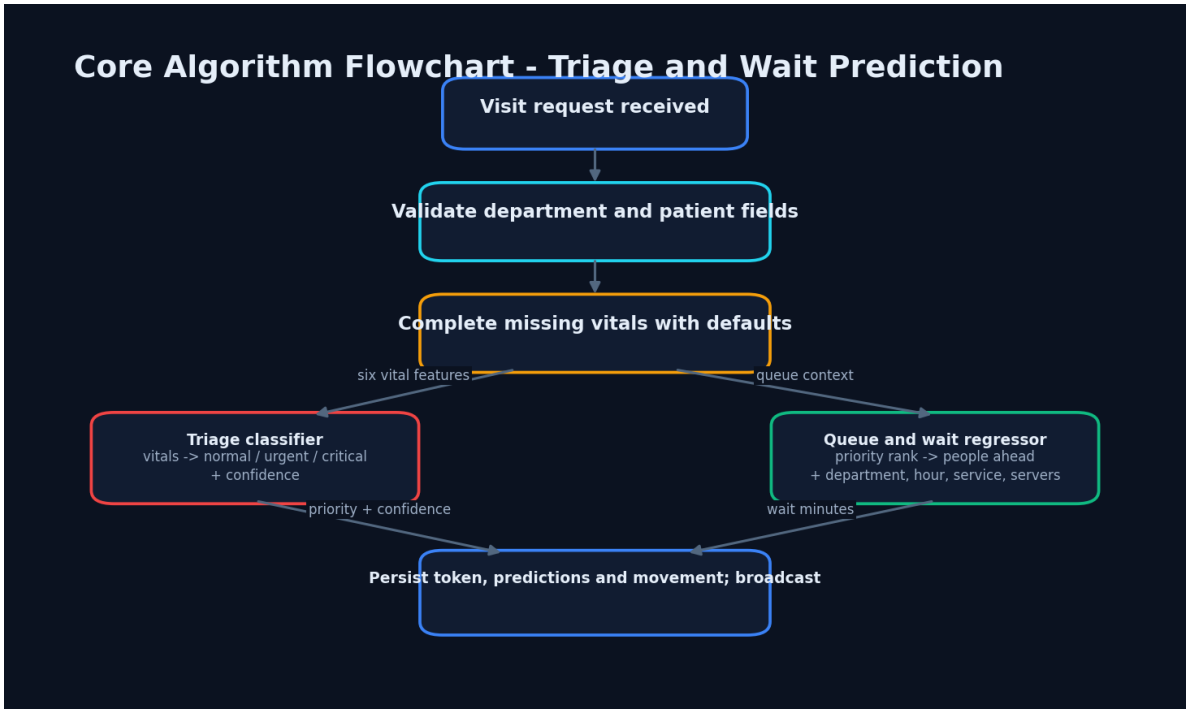


Figure 3. Prediction integration.

The predictor lazily loads Joblib bundles and automatically trains only when either artefact is missing. Department and priority encoders are stored beside the wait model. The triage bundle stores its feature order. Inference constructs pandas DataFrames with the same feature names used in training.

7. Security and Error Handling

Passwords are hashed with bcrypt through Passlib. JWTs carry subject, role and expiry and default to a 12-hour lifetime. OAuth2PasswordBearer extracts tokens and inactive users are rejected. Route-specific require_roles checks return HTTP 403 for forbidden writes; missing records return 404 and unavailable beds return 400.

Production work must replace the fallback secret, authenticate WebSockets, restrict CORS, add audit logging, enforce TLS and define privacy retention controls.

8. Frontend Design

React Router separates public landing/login/board routes from protected role homes. Axios injects the bearer token and clears local session state after HTTP 401. Zustand stores session and toast state. Role dashboards fetch REST data and refresh when matching WebSocket events arrive.

The visual system uses dark high-contrast panels, priority/status badges, reusable cards and modals. Real 1440 x 900 captures confirm the principal screens, but no formal WCAG audit was performed.

9. Key Code Walkthrough

Visit creation is the central orchestration path: resolve department; find or create patient; merge supplied values with defaults; classify priority; allocate token; count queue; predict wait; insert Visit and MovementLog; commit; publish VisitOut. The transaction intentionally stores the returned prediction so the public board and authenticated dashboards agree.

The dashboard aggregates counts and averages directly in SQLAlchemy. Bed occupancy divides occupied beds by total beds, with a denominator fallback of one to avoid division by zero.

10. Deployment and Execution

The checked-in development configuration runs Uvicorn on port 8010 and Vite on 5173; Vite proxies /api and /ws to the backend. SQLite is created in the backend working directory. Models reside under app/ml/artifacts. The repository has no run.bat, Dockerfile or production process manager.

11. Design Limitations

Both model datasets are synthetic and generated from designed distributions; they are not real patient records.

The measured model test partitions come from the same deterministic generators as training and are not independent clinical validation.

No real hospital deployment, clinician study, user-acceptance study, security audit, accessibility audit, or concurrent-load test was performed.

The 96.80% triage accuracy must be interpreted with the 63% critical-class recall; accuracy alone overstates safety-critical performance.

SQLite, a development JWT secret fallback, an unauthenticated WebSocket, and in-process broadcasting are unsuitable for production without hardening.

The landing-page claims of 42% wait reduction, 214 patients, and six departments are hard-coded marketing copy and are excluded from results.